

Agentes de búsqueda y evolutivos para TileUp

Instrucciones

Lea este documento completo antes de repartir el trabajo dentro de su grupo. Esta tarea se resuelve en grupos de dos o tres personas, sin excepción; los grupos de tres entregan además el módulo adicional descrito más adelante, que no otorga puntos extra. Esta tarea no contempla defensa, de modo que todo elemento que deba evaluarse ha de quedar demostrado en el repositorio y en el video. Las fechas oficiales son las publicadas en Discord.

1 Descripción

Esta tarea consiste en construir dos agentes que resuelvan de forma automática el juego TileUp, cuyas reglas quedan fijadas en la sección siguiente. Un agente resuelve el problema mediante un algoritmo de **búsqueda** y el otro mediante un algoritmo **evolutivo**. El grupo elige cuál usa en cada caso, lo formula sobre el problema y lo ajusta.

La secuencia de fichas de una partida es fija y se lee de un archivo de texto, de manera que el problema es determinista y completamente observable desde el primer paso. No hay azar durante la ejecución ni información oculta. El agente conoce de antemano toda la secuencia y puede planificar la partida entera antes de colocar la primera ficha.

El alcance del juego es deliberadamente reducido. No se incorporan reglas adicionales, bonificaciones ni obstáculos: solo la colocación de fichas, su agrupación por color y la suma de sus valores. Lo que se evalúa no es la riqueza del juego sino la formulación del problema, la corrección de los dos algoritmos y la calidad de la comparación experimental entre ellos.

2 Especificación del juego

Estado

El tablero es una cuadrícula de $N \times N$ celdas. Cada celda está vacía o contiene una ficha. Una ficha es un par $\langle c, v \rangle$, donde c es su color, un entero en el rango $1 \dots K$, y v es su valor, un entero positivo. El tablero comienza vacío.

Los parámetros N y K se leen de la instancia y permanecen constantes durante toda la partida. La partida dispone además de una secuencia fija de M fichas, que se consumen en el orden en que aparecen en el archivo.

Acción

En cada paso el agente toma la primera ficha pendiente de la secuencia y la coloca en **cualquier celda vacía** del tablero. No hay gravedad ni restricción de columna, de modo que el factor de ramificación de un estado con e celdas vacías es exactamente e .

Fusión

Sea p la celda donde se acaba de colocar una ficha y sea G la componente conexa por vecindad ortogonal (arriba, abajo, izquierda y derecha) formada por las fichas del mismo color que contienen a p . Si $|G| \geq 2$, todas las fichas de G se retiran del tablero y en p queda una única ficha del mismo color, cuyo valor es la suma de los valores de las fichas de G . Si $|G| = 1$, la ficha permanece donde fue colocada y el tablero no cambia de otro modo.

La fusión no encadena. Como G es la componente conexa maximal de ese color, la ficha resultante no puede quedar adyacente a otra ficha del mismo color. Nótese también que el valor total presente en el tablero no depende de las decisiones del agente, ya que la fusión conserva la suma; lo que sí depende de ellas es cuántas celdas quedan ocupadas.

Condiciones de término

La partida termina con **victoria** cuando se colocan las M fichas de la secuencia. Termina con **derrota** cuando queda al menos una ficha pendiente y el tablero no tiene ninguna celda vacía.

Objetivo

El objetivo es consumir la secuencia completa sin desbordar el tablero. Entre dos partidas que lo logran, se prefiere la que deja el tablero más despejado. El orden que decide el concurso está definido en la sección 4.

3 Núcleo obligatorio

El núcleo es obligatorio para grupos de dos y de tres personas por igual, y consta de los siguientes cinco puntos.

1. **Motor del juego.** El grupo implementa el juego descrito arriba: la representación del estado, la colocación, la fusión y la detección de victoria y derrota. El motor es independiente de los agentes y ninguno de los dos puede alterar sus reglas.
2. **Un agente de búsqueda.** El grupo elige el algoritmo, que puede ser informado o no informado. El README documenta la formulación completa, es decir, la representación del estado, el operador de sucesión, el costo de cada acción, la prueba de meta y, cuando el algoritmo sea informado, la heurística junto con el argumento de su admisibilidad o de la razón por la cual se renuncia a ella. Una heurística no admisible es aceptable si el grupo la declara y explica qué garantía pierde.
3. **Un agente evolutivo.** El grupo elige la familia del algoritmo. El README documenta la representación del individuo, la función de aptitud, el mecanismo de selección, los operadores de variación, la política de reemplazo y el criterio de paro, con los valores de los parámetros y el procedimiento por el cual se fijaron.
4. **Comparación experimental.** Los dos agentes se ejecutan sobre el mismo conjunto de instancias, que incluye al menos seis configuraciones distintas de N , K y M , y cada configuración se corre con un mínimo de tres semillas. El informe reporta, por agente y por instancia, la cantidad de fichas colocadas, las celdas ocupadas al terminar, el tiempo de cómputo y una medida de esfuerzo propia del algoritmo (nodos expandidos en el

de búsqueda, evaluaciones de aptitud en el evolutivo). Los resultados se presentan con su dispersión entre semillas, no como un número único.

5. **Validador independiente.** La entrega incluye un programa que, dada una instancia y un archivo de solución, reproduce la partida paso a paso y determina si la solución es legal, informando la cantidad de fichas colocadas y las celdas ocupadas al final. El validador no comparte código de decisión con los agentes y es la herramienta con la que se arbitra el concurso.

Además del código, el repositorio incluye un `README.md` en Markdown que permita a una persona ajena al grupo ejecutar todo sin consultas adicionales, y un informe breve, también en Markdown, con la formulación de ambos agentes y la comparación experimental.

Requisitos funcionales mínimos del sistema

El lenguaje de implementación es libre, pero el contrato del programa no lo es. Para que las entregas de distintos grupos resulten comparables y para que el concurso pueda arbitrarse, el sistema debe cumplir lo siguiente.

1. **Ejecución por línea de comandos.** Un único punto de entrada recibe la ruta de la instancia, el agente que debe usarse, la semilla y el límite de tiempo en segundos. La ejecución no requiere edición de archivos ni pasos interactivos.
2. **Lectura de la instancia.** El programa lee el formato definido más abajo. Un archivo mal formado produce un mensaje de error legible y un código de salida distinto de cero, nunca una traza de excepción sin controlar.
3. **Escritura de la solución.** El programa escribe un archivo de solución en el formato definido más abajo, aun cuando la partida termine en derrota, con las colocaciones que alcanzó a realizar.
4. **Respeto del límite de tiempo.** Alcanzado el límite indicado, el agente detiene la búsqueda y entrega la mejor solución encontrada hasta ese momento. Un agente que exceda el límite queda fuera del concurso en esa instancia.
5. **Determinismo.** Dos ejecuciones con la misma instancia, el mismo agente y la misma semilla producen la misma solución. Toda fuente de azar se deriva de la semilla recibida.
6. **Registro de métricas.** Al terminar, el programa informa por salida estándar la cantidad de fichas colocadas, las celdas ocupadas, el valor de la ficha mayor, el tiempo transcurrido y la medida de esfuerzo del algoritmo.
7. **Pruebas automatizadas.** Las pruebas unitarias verifican el motor de forma aislada, con al menos los casos de fusión de dos fichas, fusión de una componente de tres o más, colocación sin fusión y detección de derrota. Las pruebas de integración resuelven una instancia pequeña de principio a fin y comprueban con el validador que la solución producida es legal. Todo se ejecuta con un comando documentado en el `README`.

Formato de la instancia

La instancia es un archivo de texto plano. Las líneas en blanco y las que comienzan con `#` se ignoran. El resto se lee en este orden: una línea con N y K separados por espacio, una línea con M , y a continuación M líneas, cada una con el color y el valor de una ficha, en el orden en que deben colocarse.

```
# TileUp -- instancia de ejemplo
4 3      # tablero 4x4, 3 colores
6        # 6 fichas en la secuencia
```

```
1 2
2 1
1 3
3 1
1 1
2 4
```

Formato de la solución

El archivo de solución contiene una línea por colocación, en el orden de la secuencia, con tres enteros separados por espacio: el índice de la ficha comenzando en cero, la fila y la columna de la celda elegida, ambas comenzando en cero. La última línea del archivo comienza con # y resume el resultado con la cantidad de fichas colocadas, las celdas ocupadas al terminar y el valor de la ficha mayor.

```
0 0 0
1 1 1
2 0 1
3 2 2
4 0 2
5 1 2
# colocadas=6 ocupadas=3 mayor=6
```

Criterios de aceptación

Cuatro comprobaciones deciden si el núcleo está funcionando, y todas se ejecutarán tal cual sobre el repositorio recién clonado.

1. **Un solo comando.** El sistema se construye y se ejecuta con un único comando documentado, sin instalar dependencias a mano en la máquina del evaluador y sin editar archivos.
2. **Instancia no publicada.** El evaluador ejecuta una instancia que el grupo no ha visto, con el formato definido aquí, y ambos agentes producen un archivo de solución dentro del límite de tiempo indicado.
3. **La solución es legal.** El validador del propio grupo, aplicado a la instancia anterior y a la solución producida, la acepta y reporta métricas coherentes con las que el programa informó por salida estándar.
4. **Las pruebas corren.** El comando de pruebas documentado ejecuta las unitarias y las de integración, y todas pasan.

4 Concurso

Durante la sesión de revisión se ejecuta una instancia común para todos los grupos, cuya secuencia no se publica de antemano. Los valores de N , K y M sí se anuncian con antelación, de modo que cada grupo pueda dimensionar su agente. Todas las ejecuciones se realizan en la misma máquina y con el mismo límite de tiempo, y cada grupo elige cuál de sus dos agentes compite.

El resultado se ordena por tres criterios aplicados en cascada. Primero, la mayor cantidad de fichas colocadas. Segundo, ante igual cantidad de fichas, la menor cantidad de celdas ocupadas al terminar. Tercero, ante igualdad en ambos, el menor tiempo de cómputo. El arbitraje se hace con el validador del grupo evaluado, y una solución que el validador rechace se descarta.

El primer lugar recibe cinco puntos extra sobre la nota de esta tarea y el segundo recibe tres. Estos puntos se suman fuera de los cien de la rúbrica, no sustituyen ningún criterio y no compensan la ausencia de ninguno de los componentes exigidos.

5 Regla de frontera

Se prohíbe delegar en una biblioteca o en un tercero la construcción de aquello que constituye el objeto de aprendizaje. En esta tarea la frontera se ubica en los dos algoritmos y en el motor del juego, de manera que **la búsqueda, el algoritmo evolutivo y las reglas del juego deben ser escritos por el grupo**.

En consecuencia, no se acepta el uso de bibliotecas que implementen la búsqueda en grafos o en espacios de estados, ni de marcos de computación evolutiva o de optimización metaheurística, ni de resolvedores de restricciones, de programación entera o de satisfacibilidad, ni de un motor del juego tomado de un tercero. Tampoco se acepta invocar un modelo de lenguaje en tiempo de ejecución para que decida las colocaciones. Un componente que el grupo no sepa fundamentar no se contabiliza, aunque funcione.

Sí se permite todo lo que pertenece a capas inferiores del problema, a saber, las estructuras de datos y las utilidades de la biblioteca estándar del lenguaje (colas de prioridad, colas dobles, generadores de números aleatorios, tablas de dispersión), las bibliotecas de cálculo numérico y de gráficos para el informe, y los marcos de pruebas. Contenerizar, medir tiempos y graficar resultados no forma parte de la frontera.

6 Módulo adicional para grupos de tres

Los grupos de tres personas añaden un estudio empírico de escalabilidad. Se requiere un generador de instancias parametrizado por N , K y M , y por una semilla, que produzca archivos en el formato definido en esta especificación. Sobre él se construye una batería experimental que recorra al menos tres valores de N y tres de K , con un mínimo de tres semillas por configuración.

El informe reporta cómo se degradan ambos agentes al crecer el tablero y al crecer la cantidad de colores, con la dispersión entre semillas y con al menos una gráfica que muestre la tendencia. Se espera una lectura del resultado, no solo la tabla: qué parámetro domina el costo, en qué punto el agente de búsqueda deja de terminar dentro del límite de tiempo y cómo se comporta el evolutivo en ese mismo régimen.

Este módulo **no otorga puntos extra**, ya que constituye el alcance esperado de un grupo de tres personas. Un grupo de tres que entregue solo el núcleo se evalúa con este módulo en cero.

7 Entrega

La tarea se publica en la Semana 8 y se entrega el jueves de la Semana 10, a más tardar a las 10:00 p.m. por el buzón de TecDigital; la fecha oficial es la publicada en Discord. Cada día de atraso descuenta cinco puntos sobre base cien. Esta tarea no contempla defensa, de modo que la evaluación se realiza exclusivamente sobre lo que el repositorio y el video demuestren.

La entrega en TecDigital consiste en un único paquete que contiene:

1. El enlace al repositorio de GitHub. El historial debe mostrar *commits* de todos los integrantes; la autoría individual se determina por los *commits* y, en los archivos comunes, se considera compartida. Un integrante sin *commits* se evalúa individualmente según la autoría que pueda demostrarse.
2. El código fuente completo, que comprende el motor, los dos agentes, el validador, las pruebas y, en los grupos

de tres, el generador de instancias y los guiones de la batería experimental. El repositorio no debe contener dependencias instaladas, de modo que quedan fuera `node_modules`, `.venv`, `vendor` y equivalentes.

3. El conjunto de instancias con el que se hizo la comparación experimental, junto con las soluciones producidas, para que los resultados del informe puedan verificarse con el validador.
4. La documentación, en formato **Markdown** dentro del repositorio, que comprende el `README.md` con las instrucciones reproducibles y el informe con la formulación de ambos agentes, sus parámetros y la comparación experimental. No se aceptan documentos en Word ni PDF como sustituto.
5. Un video de demostración de máximo diez minutos, **sin edición**, que muestre la clonación del repositorio, la ejecución de ambos agentes sobre una instancia, la validación de las soluciones producidas y el recorrido del informe. El video tiene un fin práctico, ya que evita que la evaluación se bloquee por un problema de entorno en la máquina del evaluador.

8 Evaluación

La evaluación distingue el *criterio técnico demostrable* (70 puntos), que recoge lo que puede verse funcionando con evidencia reproducible, y la *profesionalidad* de la entrega (30 puntos). Cada criterio tiene un peso, y la nota de cada uno resulta de multiplicar ese peso por el nivel de logro alcanzado según la matriz. En una tarea corta no se exige documentación a profundidad, pero sí que lo entregado esté ordenado, se entienda y corra.

Conforme al Principio de Defensa Técnica, un componente cuya justificación no conste en el informe se evalúa en el nivel que corresponda a la evidencia disponible, aunque el programa produzca el resultado esperado.

Rúbrica de la Tarea Corta 1

Criterio (peso)	Excelente (100 %)	Bueno (75 %)	Regular (50 %)	Deficiente (≤ 25 %)
Motor y validador (12)	Reglas implementadas sin desviaciones, validador independiente que acepta y rechaza correctamente, cubierto por pruebas unitarias.	Motor correcto y validador funcional, con cobertura de pruebas incompleta.	Fusión o detección de derrota con errores en casos límite, o validador que solo reproduce la partida sin verificar legalidad.	Motor que no respeta las reglas, o ausencia de validador.
Agente de búsqueda (16)	Formulación completa y correcta del estado, los operadores, el costo y la meta; heurística justificada; el agente resuelve las instancias dentro del límite.	Formulación correcta con una justificación parcial, o desempeño flojo en las instancias mayores.	Implementación funcional pero con la formulación sin documentar, o heurística sin argumento alguno.	No resuelve, o el algoritmo entregado no corresponde con el descrito.
Agente evolutivo (16)	Representación, aptitud, selección, variación y paro definidos y justificados; parámetros fijados con un procedimiento explícito; resultados reproducibles con la semilla.	Diseño correcto con parámetros elegidos sin procedimiento documentado.	Funciona, pero con componentes sin describir o sin control de la semilla.	No converge a soluciones legales, o reimplementa una búsqueda exhaustiva bajo otro nombre.
Comparación experimental (14)	Seis configuraciones o más, tres semillas por configuración, métricas completas y dispersión reportada; la lectura de los resultados es correcta.	Cubre lo pedido con alguna métrica ausente o con una lectura superficial.	Pocas instancias, una sola semilla, o resultados presentados sin análisis.	Sin comparación, o cifras que el repositorio no permite reproducir.
Escalabilidad (12, solo grupos de 3)	Generador parametrizado, batería en N y K con tres semillas, gráfica y lectura del régimen en que cada agente deja de servir.	Batería completa con análisis limitado, o sin gráfica.	Generador presente y experimentos parciales.	No se implementó.
README e informe (12)	Markdown claro con requisitos, comandos de ejecución y de pruebas, formatos de entrada y salida, y el informe con ambas formulaciones; un tercero lo sigue sin ayuda.	Cubre lo esencial, con alguna omisión.	Instrucciones incompletas o informe que no documenta las decisiones.	Ausente o insuficiente para ejecutar.
Reproducibilidad (8)	Un solo comando construye y ejecuta; la semilla fija el resultado; el límite de tiempo se respeta; los errores de entrada se manejan.	Reproducible con un paso manual menor.	Requiere ajustes en la máquina del evaluador, o el determinismo falla.	No se logra ejecutar sin intervención del grupo.
Repositorio y autoría (5)	Historial con commits de todos, estructura ordenada y sin dependencias versionadas.	Commits de todos, con algún descuido de orden.	Contribución desigual o dependencias versionadas.	Un solo autor, o repositorio desordenado.
Video y entrega (5)	Video de máximo diez minutos sin edición que muestra el flujo completo; entrega conforme.	Video que cubre casi todo el flujo.	Video parcial, o incumple algún requisito de entrega.	Sin video, o no demuestra el sistema.

Grupos de dos. El criterio de escalabilidad no aplica y sus 12 puntos se redistribuyen en el bloque técnico, con lo que motor y validador pasa a 14, agente de búsqueda a 20, agente evolutivo a 20 y comparación experimental a 16. Un grupo de tres que no entregue ese módulo obtiene el nivel Deficiente en él.

Uso de inteligencia artificial. El uso de herramientas de IA está permitido, pero cada entrega incluye una breve declaración de cómo se usaron, en qué partes y qué se verificó de forma manual. Declararlo no penaliza; ocultarlo agrava. Si en la revisión no se distingue la aportación intelectual del grupo, porque el trabajo aparente haber sido producido de forma exclusiva por una herramienta automática, la entrega queda sujeta a penalización, según el Principio de Aportación Demostrable y la cláusula BDFL de las disposiciones del curso.

Aplican además las penalizaciones transversales del curso. Se descuentan cinco puntos por cada día de atraso sobre base cien. La ausencia de *commits* de un integrante convierte su evaluación en individual, según la autoría que pueda demostrarse. Subir al repositorio las carpetas de dependencias ya instaladas (`node_modules`, `.venv`, `vendor` y equivalentes), en lugar de excluirlas con `.gitignore`, descuenta cinco puntos del bloque de profesionalidad.